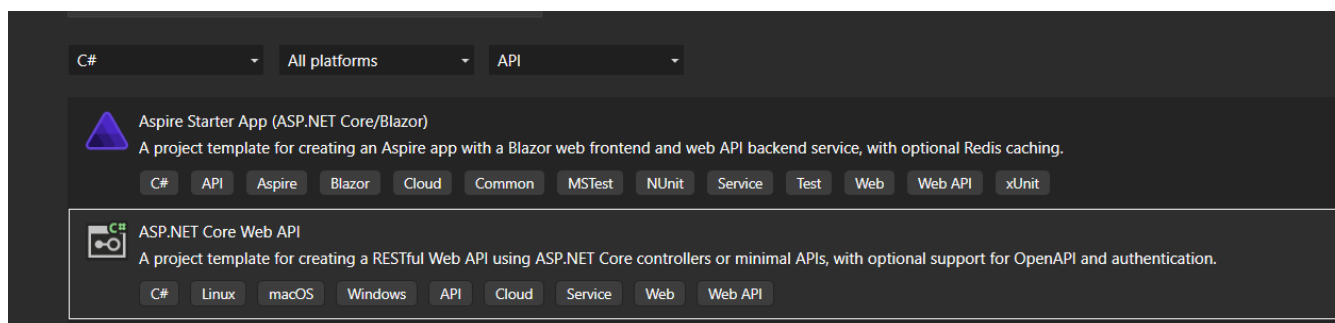


# Övning 1 API – Entity Framework Core - Lagersystem - StorageApi

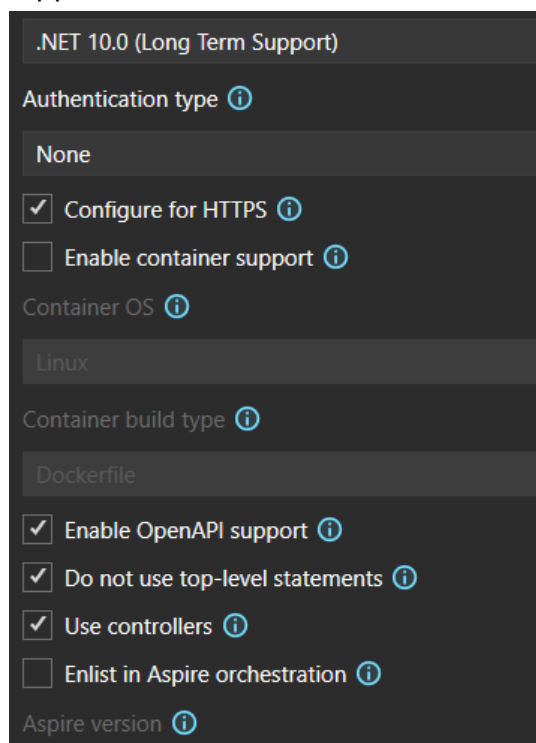
Ladda ner och installera Postman (<https://www.postman.com/downloads/>) som används för att **testa API: er under utveckling**.

## Del 1 – Skapa projektet

1. Skapa ett nytt projekt: ASP.NET Core Web API (.NET 10)



2. Namnge projektet StorageApi
3. Välj Authentication type: None, se till att Use controllers och Enable OpenApi support är markerat



4. Add Source-control – Git I Visual Studio

5. Lägg till följande i .gitignore  
# Visual Studio & byggfiler  
bin/  
obj/  
.vs/  
\*.user  
\*.suo  
\*.sln.docstates
6. git add .gitignore  
git commit -m "add Visual studio byggfiler"  
git push -u origin
7. git remote add origin klassens repo
8. git push -u origin master

## Del 2 – Skapa din modell

Använd branches, feature/model-storage.

git checkout -b feature/model-storage

Lägg till modellkod, commit: git commit -m "Add Storage model"

Push: git push -u origin feature/model-storage

Testa/PR till main när klart

1. Skapa en mapp **Models**
2. Skapa en **klass Product.cs** med följande egenskaper:  
int Id  
string Name  
int Price  
string Category  
string Shelf  
int Count  
string Description

git checkout master lokalt // Växla till master lokalt

git pull // Ta hem senaste från GitHub:s master till lokalt master

git branch -D feature/model-storage // Radera branch feature/model-storage

## Del 3 – Skapa Controller

Använd branches, feature/controller-storage

`git checkout -b feature/controller-storage`

Skapa en ProductsController, under mappen Controllers, commit: `git commit -m "Add controller-Storage"`

Push: `git push -u origin feature/controller-storage`

Testa/PR till main när klart

1. Högerklicka på mappen Controllers och välj Add > New Scaffolded Item
2. Välj API Controller with actions, using Entity Framework
3. Välj Product som modellklass och skapa upp ett nytt StorageContext som datakontextklass (tryck på + tecknet)
4. Döp kontrollern till ProductsController och tryck OK för att generera koden

`git checkout master` lokalt // Växla till master lokalt

`git pull` // Ta hem senaste från GitHub:s master till lokalt master

`git branch -D feature/controller-storage` // Radera branch feature/controller-storage

## Del 4 – Skapa en Migration och Databas

Använd branches, feature/migration-database

git checkout -b feature/migration-database

1. Öppna Package Manager Console
2. Kör kommandot: Add-Migration Init
3. Kör sedan: Update-Database
4. Verifiera att databasen skapats i SQL Server Object Explorer
5. Lägg in några produkter manuellt i databasen

## Del 5 Efter Scaffold

Undersök koden som har skapats: Vilka metoder har genererats?

Hur används StorageContext?

Hur fungerar CreatedAtAction, Ok, NotFound osv?

**Tips:** Använd Git Changes för att granska exakt vad scaffolden genererat. Detta ger en tydlig översikt över ändringarna i projektet och hjälper dig att förstå koden bättre. (Migrations foldern genererades i del 4)

Använd Postman för att testa dina endpoints: POST /api/products – skapa produkt

GET /api/products – hämta alla produkter

GET /api/products/{id} – hämta en produkt

**Tips:** Debugga koden och se vad som händer!

---

## Del 6 – Skapa DTOs

1. Skapa en mapp DTOs

Skapa: **CreateProductDto** med samma fält som modellen (för input exklusive id)

**ProductDto** som visar Id, Name, Price, Count, samt ett beräknat fält **InventoryValue** => **Price \* Count**

Lägg till valideringsattribut som [Required], [Range] m.m.

## Del 7 - Använd DtOs

1. Refaktorera för att använda dina egna DTO-klasser istället för att returnera hela entiteten **Product**. Detta ger bättre kontroll över vilken data som skickas till klienten och gör API:et tydligare.

## Del 8 – Lägg till Swagger UI

1. Lägg till Nugetpaketet `Swashbuckle.AspNetCore.SwaggerUi`
2. Lägg till i Program
3. Lägg till i `launchSettings.json`

Utforska Swagger för att testa dina endpoints: `GET /api/products` – hämta alla produkter

`GET /api/products/{id}` – hämta en produkt

## Del 9 -Mer funktionalitet

Skapa en endpoint `GET /api/products/stats` som returnerar: Totalt antal produkter

Totalt lagervärde

Snittpris

Lägg till en funktionalitet för att kunna söka på kategorier via `querystring`

`GET /api/products/?category=KategoriNamn`

Valfritt:

- Skapa filtrering för produktnamn
- Kombinera kategori och namn i sökningen